

Знакомство с языком программирования Rust

Полное руководство по установке,
настройке и основам разработки

Установка Rust на Windows

Нам потребуется компилятор. Rust генерирует код с помощью LLVM, но для Windows ему нужен набор сборщика Microsoft C++ (MSVC).



1. Перейдите на страницу загрузки Microsoft C++ Build Tools.
2. Скачайте и запустите установщик Build Tools for Visual Studio.
3. В установщике выберите рабочую нагрузку: «Разработка классических приложений на C++».
4. Нажмите «Установить» (потребуется место на диске и время).

Windows: Устанавливаем Rust



1. Перейдите на официальный сайт rust-lang.org.
2. Скачайте rustup-init.exe (для Windows).
3. Запустите скачанный файл. Откроется черное окно терминала.
4. Вас спросят о типе установки. Введите 1 (по умолчанию) и нажмите Enter.
5. После завершения установка будет готова.

Проверка установки

Откройте новый терминал (PowerShell, Cmd или любой другой) и введите следующие команды для проверки версий:

```
rustc --version
```

Должен вывести версию компилятора, например: rustc 1.7x.x

```
cargo --version
```

Должен вывести версию пакетного менеджера Cargo

Установка на Linux / macOS

Процесс еще проще. Главное преимущество — не нужно ставить тулчейны вручную, как на Windows.

Для Linux (Ubuntu/Debian):

Сначала обновите пакеты и установите базовый набор разработчика (если не установлен):

```
sudo apt update  
sudo apt install build-essential curl
```

Установка на Linux / macOS

Для macOS

Убедитесь, что установлены Xcode command line tools:

```
xcode-select --install
```

Финальный шаг

Скопируйте и выполните главную команду с сайта:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs |  
sh
```

Следуйте инструкциям и перезагрузите терминал

Почему VS Code?

VS Code — это легкий, быстрый и настраиваемый редактор. С помощью плагинов он превращается в мощную IDE для Rust.

Что мы хотим получить:

IntelliSense: Умные подсказки при наборе кода.

Проверка синтаксиса: Подчеркивание ошибок красным прямо во время печати (до компиляции).

Автоформатирование: Красивый код одной кнопкой.

Запуск в один клик: Кнопка "Run" для быстрого старта.

Базовый набор расширений

Откройте VS Code, перейдите на вкладку Extensions (иконка квадратиков или Ctrl+Shift+X). Найдите и установите следующие расширения:

rust-analyzer — СЕРДЦЕ всего. Обеспечивает автодополнение, переход к определению, подсветку синтаксиса и проверку ошибок на лету .

CodeLLDB — Отладчик. Позволит нам ставить точки останова и смотреть значения переменных .

Even Better TOML — Красивая работа с файлом Cargo.toml (файл зависимостей проекта).

crates — Помогает обновлять версии зависимостей прямо в Cargo.toml .

Error Lens — Показывает ошибки прямо на строке кода, а не только в "проблемах".

Тонкая настройка (Settings.json)

Нажмите Ctrl+Shift+P, введите "settings json" и выберите "Preferences: Open Settings (JSON)". Добавьте эти настройки для автоматического форматирования при сохранении:

```
{
  "[rust]": {
    "editor.defaultFormatter": "rust-lang.rust-analyzer",
    "editor.formatOnSave": true
  },
  "[toml]": {
    "editor.defaultFormatter": "tamasfe.even-better-toml",
    "editor.formatOnSave": true
  },
  "rust-analyzer.inlayHints.typeHints.enable": true,
  "rust-analyzer.inlayHints.parameterHints.enable": true,
  "rust-analyzer.checkOnSave": true,
  "rust-analyzer.cargo.allFeatures": true
}
```

Создаем первый проект

Cargo — это система сборки и менеджер пакетов Rust. Мы будем использовать его для создания проекта.

Откройте терминал в VS Code (Ctrl+`) и выполните:

```
cargo new my_first_project  
cd my_first_project
```

Cargo создаст папку с файлом Cargo.toml (описание проекта) и папкой src с файлом main.rs.

Создаем первый проект

Чтобы запустить проект:

```
cargo run
```

Вы увидите вывод "Hello, world!".

Hello, World! и макросы

Откройте `src/main.rs`. Вы увидите:

```
fn main() {  
    println!("Hello, world!");  
}
```

1. `fn main()` — точка входа в программу (как в C++).
2. `println!` — это макрос. В Rust макросы отличаются восклицательным знаком `!`. Они генерируют дополнительный код. `println!` выводит текст в консоль.
3. Строки кода заканчиваются точкой с запятой `;`.

Переменные и неизменяемость

В Rust переменные по умолчанию неизменяемы (immutable). Это защищает от случайных изменений.

```
fn main() {  
    let x = 5;  
    println!("x = {}", x);  
    // x = 6; // ОШИБКА! Нельзя изменить неизменяемую переменную.  
  
    let mut y = 10; // Ключевое слово mut делает переменную изменяемой  
    println!("y = {}", y);  
    y = 15; // А так можно  
    println!("y теперь = {}", y);  
}
```

Типы данных (Скалярные)

Rust — статически типизированный язык. Компилятор часто сам понимает тип (type inference), но мы можем указать его явно.

```
fn main() {  
    // Целые числа  
    let a: i32 = -100; // 32-битное целое со знаком  
    let b: u64 = 1000; // 64-битное целое без знака  
    let c = 255; // Компилятор поймет, что это i32 по умолчанию  
  
    // Числа с плавающей точкой  
    let d: f32 = 3.14;  
    let e = 2.718; // f64 по умолчанию  
  
    // Логический тип  
    let flag: bool = true;  
    let is_greater = 10 > 5; // false  
  
    // Символ (Unicode)  
    let heart_eyed_cat: char = '🐱';  
}
```

Типы данных (Составные)

1. Кортеж (Tuple): Собирает вместе значения разных типов. Имеет фиксированный размер.
2. Массив (Array): Все элементы одного типа. Фиксированный размер (хранится в стеке).
3. Ссылки и заимствование (References & Borrowing)

Типы данных (Составные)

```
fn main() {  
    // Кортеж  
    let person: (&str, i32, char) = ("Alice", 30, '👤');  
    // Доступ к элементу по индексу (точка)  
    println!("Имя: {}", person.0);  
  
    // Массив  
    let numbers = [1, 2, 3, 4, 5];  
    let first = numbers[0]; // Индексация с нуля  
  
    // Вектор (динамический массив из стандартной библиотеки) - более частый  
    // аналог std::vector  
    let mut vec = Vec::new();  
    vec.push(10);  
    vec.push(20);  
    println!("Вектор: {:?}", vec);  
}
```

Управляющие конструкции (if/else)

В Rust `if` — это выражение. Оно может возвращать значение. Условие всегда должно быть `bool` (никакого `if(5)` как в C++!)

Управляющие конструкции (if/else)

```
fn main() {  
    let number = 7;  
  
    if number < 5 {  
        println!("Меньше пяти");  
    } else if number == 5 {  
        println!("Равно пяти");  
    } else {  
        println!("Больше пяти");  
    }  
  
    // if в присваивании (так как это выражение)  
    let condition = true;  
    let value = if condition { 50 } else { 100 };  
    println!("value = {}", value); // 50  
}
```

ЦИКЛЫ

Три вида циклов:

1. `loop` — бесконечный цикл, пока его не прервать.
2. `while` — цикл с условием.
3. `for` — самый используемый для перебора коллекций.

```
fn main() {  
    // loop  
    let mut counter = 0;  
    let result = loop {  
        counter += 1;  
        if counter == 10 {  
            break counter * 2; // break может вернуть значение  
        }  
    };  
    println!("Результат loop: {}", result); // 20  
  
    // while  
    let mut n = 3;  
    while n != 0 {  
        println!("{}", n);  
        n -= 1;  
    }  
  
    // for (работает с итераторами)  
    let arr = [10, 20, 30];  
    for element in arr.iter() {  
        println!("Значение: {}", element);  
    }  
    // Диапазон (1..4) не включает 4  
    for i in 1..4 {  
        println!("i = {}", i); // 1, 2, 3  
    }  
}
```

Функции

Функции объявляются через `fn`.
Тип возвращаемого значения указывается после `->`. В теле функции последнее выражение (без `;`) является возвращаемым значением. Можно использовать `return` досрочно

```
// Функция, принимающая i32 и возвращающая i32
fn add_five(x: i32) -> i32 {
    // Это выражение, оно возвращает результат
    (нет ';' )
    x + 5
    // Если бы мы написали x + 5; - это была бы
    инструкция, и функция вернула бы ()
}

fn main() {
    let sum = add_five(10);
    println!("Результат: {}", sum); // 15

    let sum2 = add_or_subtract(10, true);
    println!("Результат 2: {}", sum2);
}

fn add_or_subtract(x: i32, add: bool) -> i32 {
    if add {
        x + 5 // Возврат из if
    } else {
        x - 5
    }
}
```

Структуры (struct)

```
// Объявляем структуру
struct User {
    username: String,
    email: String,
    sign_in_count: u64,
    active: bool,
}

fn main() {
    // Создаем экземпляр
    let user1 = User {
        email: String::from("user@example.com"),
        username: String::from("bestuser"),
        active: true,
        sign_in_count: 1,
    };

    println!("Email пользователя: {}", user1.email);

    // Чтобы изменить поле, весь экземпляр должен быть mut
    let mut user2 = User { ..user1, email: String::from("new@mail.ru") }; // Синтаксис обновления
    user2.active = false;
```

Ссылки и заимствование

В Rust нет классических указателей, как в C/C++. Вместо этого используются ссылки, которые всегда гарантированно указывают на валидные данные. Ссылки бывают двух типов:

1. Неизменяемые (`&T`) — позволяют только читать данные.
2. Изменяемые (`&mut T`) — позволяют изменять данные, но только одна такая ссылка может существовать в области видимости (предотвращает гонки данных).

Правила заимствования (Borrowing):

1. В любой момент времени у вас может быть либо одно изменяемое заимствование, либо любое количество неизменяемых, но не одновременно.
2. Ссылка всегда должна быть валидной (компилятор проверяет время жизни).

Ссылки и заимствование

```
fn main() {  
    let mut s = String::from("hello");  
  
    let r1 = &s;          // неизменяемая ссылка  
    let r2 = &s;          // ещё одна неизменяемая – ок  
    println!("{}", r1, r2);  
  
    let r3 = &mut s;      // изменяемая ссылка (предыдущие r1, r2  
    вышли из области видимости)  
    r3.push_str(", world");  
    println!("{}", r3);  
}
```


Отладка (Debugging)

Благодаря ранее установленному расширению CodeLLDB, отладка работает "из коробки" для простых случаев.

1. Откройте файл `main.rs`.
2. Слева от номеров строк нажмите левой кнопкой мыши — появится красная точка (breakpoint).
3. Перейдите на вкладку Run and Debug (иконка с жучком и треугольником) или нажмите `Ctrl+Shift+D`.
4. В выпадающем списке выберите "Debug (LLDB)" (если есть) или нажмите "create a launch.json file" и выберите LLDB. Обычно автоматически создается конфиг.
5. Нажмите зеленую стрелочку "Start Debugging" (F5).

Работа с точками останова

Рассмотрим код с циклом и условием.

```
fn main() {  
    let mut values = vec![1, 2, 3, 4, 5];  
    let mut sum = 0;  
  
    for value in &values {  
        sum += value;  
        if sum > 5 {  
            println!("Сумма стала больше 5: {}", sum);  
        }  
    }  
    println!("Итоговая сумма: {}", sum);  
}
```

Работа с точками останова

1. Простая точка останова: Поставьте точку на строке `sum += value;`.
2. Условная точка останова: Кликните правой кнопкой по красной точке -> "Edit Breakpoint...". Введите условие, например `value == 3`. Отладка прервется только когда `value` будет равен 3.
3. Просмотр переменных: Когда программа остановится, в левой панели VS Code (панель Variables) вы увидите текущие значения `values`, `sum`, `value`.
4. Панель инструментов: Используйте кнопки: Continue (F5), Step Over (F10), Step Into (F11), *Step Out (Shift+F11)* для построчного выполнения.

Rust vs C++

Ключевое преимущество: Memory Safety без GC

C++ дает полную свободу, но вы можете стрелять себе в ногу. Rust предотвращает это на этапе компиляции благодаря Ownership Model (Владение) и Borrow Checker (Проверка заимствований).

Чего нет в Rust (или есть, но компилятор не даст им случиться):

Разыменование нулевых указателей (null pointer dereferencing). В Rust нет NULL, есть `Option<T>`.

Висячие указатели (dangling pointers).

Двойное освобождение памяти (double free).

Гонки данных (data races) в многопоточности (гарантируется на этапе компиляции!)

Современный инструментарий

C++:

1. Нет стандартного менеджера пакетов (используем `vsrkg`, `Conan` или ручной `git clone`).
2. Система сборки: `CMake`, `Make`, `MSBuild`... Очень много вариантов и боли.
3. Форматирование: нужно отдельно настраивать `clang-format`.

Rust:

1. `Cargo`: Единый стандарт. `cargo new`, `cargo build`, `cargo run`, `cargo test`, `cargo doc`.
2. `crates.io`: Гигантский репозиторий библиотек. Добавить зависимость — одна строка в `Cargo.toml`.
3. `rustfmt`: Единый стандарт форматирования кода (команда `cargo fmt`).
4. `Clippy`: Линтер с сотнями советов по улучшению кода (`cargo clippy`).
5. Понятные ошибки: Компилятор Rust славится дружелюбными сообщениями об ошибках, которые часто предлагают способ исправления .

Производительность без компромиссов

- Rust — это компилируемый язык, использующий LLVM (как и Clang для C++). Это значит, что по скорости он на равных с C++.
- Отсутствие сборщика мусора (Garbage Collector) означает предсказуемую задержку (no GC pauses), что критично для игр, встраиваемых систем и высоконагруженных сервисов.
- Абстракции с нулевой стоимостью (zero-cost abstractions), как в C++: использование итераторов, дженериков (generics) не медленнее ручного написания циклов .